

Build the Docker image from your project directory (where this Dockerfile is):

```
docker build -t my-java-app .
```

Run the Docker container and map port 8080 (adjust if your app uses a different port):

```
docker run -p 8080:8080 my-java-app
```

Now your Java app runs inside a container and is accessible at <http://localhost:8080>.

Building and running the Docker image

Once you've set up your Dockerfile and your Java application is neatly packaged as a JAR, the next step is to build a Docker image and run it as a container. Building the image means that Docker takes all the instructions from your Dockerfile and creates a snapshot that includes your Java app along with the necessary Java runtime environment. Think of this image as a portable package that you can run on any machine with Docker installed, without having to stress about differences in the environment. To build the image, you'll need to run a Docker command in your project directory. This command reads your Dockerfile, copies your Java app into the image, and sets everything up just the way you specified. Once the build is complete, you'll have an image named whatever you decided to tag it — for instance, *my-java-app*.

When it comes to running the Docker container, you're essentially launching an instance of that image. As the container starts up, it runs your Java app in a controlled and isolated environment. You can also map the container's internal ports to your computer's ports, allowing you to access your app from a browser or API client as if it were running directly on your machine. Lastly, you can easily check which containers are currently running with a simple Docker command. This is a great way to keep an eye on your apps and manage them effectively.

Using Docker to build and run your Java application guarantees that it behaves the same way everywhere, making development, testing, and deployment a whole lot easier and more reliable.

To build the Docker image from the current directory, tag it as *my-java-app*:

```
docker build -t my-java-app
```

Run a container from the image, mapping container port 8080 to host port 8080:

```
docker run -p 8080:8080 my-java-app
```

List all running containers to verify your app is

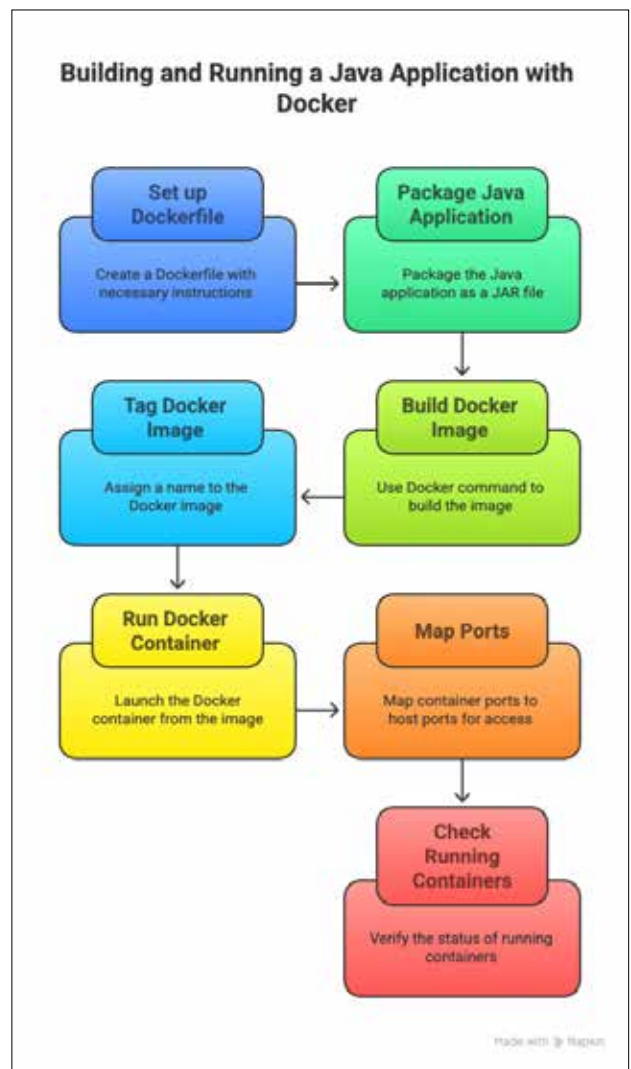


Figure 2: Building and running a Java application with Docker running (optional):

```
docker ps
```

Multi-stage Docker builds (optional, for optimised images)

When you're putting together a Docker image for your Java application, it typically includes all the necessary files and tools for both building and running the app. However, this can lead to a hefty final image size since build tools like Maven or Gradle, along with the source code, don't really need to be part of the image that's deployed in production. That's where multi-stage builds come in handy. They allow you to use one stage to compile and build your Java app, and then you can just copy the final runnable files (like the JAR) into a much smaller image that only has what's essential for running the app. The end result? A leaner, cleaner Docker image that takes